



HagerGroup



UIMM Alsace

Revue 1 : présentation d'entreprise

Apprenti : Timothée Delhelle

Tuteur : Nicolas Freyd

Diplôme :

BTS CIEL (Cybersécurité, Informatique et Réseaux,
Electronique)

Promotion :

2023-2025

Remerciements

Je remercie Nicolas Freyd, mon tuteur, ainsi que Christian Kuhn, mon supérieur pour m'avoir donné l'opportunité de réaliser mon BTS en alternance au sein de l'entreprise Hager.

Mes remerciements vont également à tous mes collègues qui, au travers de leur bienveillance, leur savoir-faire et leur disponibilité, ont fait en sorte que mon alternance se déroule dans les meilleures conditions.

Enfin, je remercie les enseignants du Pôle Formation UIMM Alsace pour leur enseignement de qualité.

Table des matières

1	Présentation de l'entreprise	4
1.1	Histoire de l'entreprise	4
1.2	Localisation	5
1.2.1	Global	5
1.2.2	Site principale	5
1.3	Secteurs d'activité	6
1.4	Produits phares	6
1.5	Chiffres clés	7
1.6	Organisation des services informatiques	8
1.7	Présentation de l'équipe	9
2	Les missions de l'apprenti	10
2.1	Sauvegarde du parc informatique	10
2.2	Formation des utilisateurs	11
2.3	Cartographie du réseau industriel	12
2.4	Provisioning de postes	13
2.5	Migration de PC de production	13
2.6	Ecriture de scripts	14
2.6.1	Script de purge de disque :	14
3	Projet E6	15
3.1	Gestion de projet	15
3.1.1	Diagramme de contexte :	15
3.1.2	Diagramme UML de cas d'utilisation :	16
3.1.3	Diagramme de séquence de visualisation des données :	17
3.1.4	Diagramme de séquence de réception / envoi des données :	18
3.1.5	Diagramme de Gantt	18
3.1.6	Structure interne de la base de données	19
3.2	Matériels utilisés	19
3.3	Logiciels Utilisés	20
3.4	NodeRed	21
3.4.1	Flow 1 :	21
3.4.2	Requête SQL de récupération des données	22
3.4.3	Affichage des données	23
3.4.4	Code de conversion des données de la base de données pour l'affichage	24
3.4.5	CSS Custom	25
3.4.6	Flow 2 :	27

3.4.7	Authentification :	27
3.4.8	Interface de visualisation des données :	28
3.5	Script de transfert des données depuis MQTT vers MariaDB	29
3.6	MariaDB	31
3.6.1	Requête SQL de création de la table d'un capteur	32
3.7	Broker MQTT	33
3.7.1	Récupération de l'image Eclipse	33
3.7.2	Premier lancement du conteneur	33
3.7.3	Lancement du conteneur après arrêt	33
3.7.4	Configuration Broker MQTT	33
3.8	Lancement automatisé des conteneurs et scripts	34
3.8.1	Service de lancement des conteneurs	34
3.8.2	Script de lancement des conteneurs	34
3.9	Firmware ESP32	35
3.9.1	Matériel Utilisé :	35
3.9.2	Activation de la puce PHY	35
3.9.3	Configuration de la MAC	36
3.9.4	Installation du driver Ethernet	37
3.9.5	Démarrage de la carte Ethernet	37
3.9.6	Event Handler du driver Ethernet	38
3.9.7	Configuration de la pile IP Netif	39
3.9.8	Liaison du driver Ethernet et de la pile IP	39
3.9.9	Configuration IP de la carte	40
3.9.10	Event Handler de la pile IP Netif	41
3.9.11	Configuration MQTT et démarrage du client	42
3.9.12	Event Handler MQTT	43
3.10	Capteur MQ135	44
3.10.1	Gaz détectés :	44
4	Conclusion	46

1 Présentation de l'entreprise

1.1 Histoire de l'entreprise

En 1955, Oswald Hager et Hermann Hager (1928-2014) créent avec leur père Peter Hager l'entreprise Hager oHG, elektrotechnische Fabrik, à Ensheim (Allemagne). À cette époque et depuis 1945, la Sarre est économiquement rattachée à la France et le marché allemand n'est pas accessible. Hager souhaite néanmoins s'implanter sur ces deux marchés.

En 1959, la famille Hager crée à Obernai, en Alsace, l'entreprise Hager Electro SA qui est la première filiale étrangère. À partir de 1966, Hager forme des installateurs électriciens de manière systématique ce qui instaure une culture de fidélisation des clients en leur fournissant des informations utiles. Le disjoncteur modulaire de Hager est breveté en Allemagne en 1968 puis en France en 1970. Dans le même temps, le premier tableau électrique en série, le « Hager-Rapid-System », fait son apparition sur le marché français. En 1973, Hager réalise un chiffre d'affaires de 43 millions de deutsche marks en Allemagne, et, en 1974, de 22 millions de francs en France.

En 1976, Hager introduit sur le marché le petit coffret de distribution Gamma. Puis, en 1982, débute la production de disjoncteurs différentiels. Un nouveau site de production avec un entrepôt à hauts rayonnages est également créé à Blieskastel. Dans les années 1980, le groupe Hager commence à se positionner comme un fournisseur de services complets pour les installations électriques dans les bâtiments. Des sociétés de distribution sont créées en Europe (Suisse, Grande-Bretagne). Depuis le milieu des années 1990, le groupe Hager a développé son réseau de distribution aux Émirats arabes unis (Dubai), à Singapour, en Malaisie, à Hong Kong, en Chine, en Australie et en Nouvelle-Zélande.

Par la suite, de nombreuses acquisitions d'entreprises par HagerGroup permettent une diversification des activités et assurent ainsi la prospérité du groupe.



Inauguration du site d'Obernai



Agrandissement du site d'Obernai

1.2 Localisation

1.2.1 Global

Hager Group est une entreprise internationale avec des sites de production présents en Europe et en Asie et une distribution des produits dans plus de 100 pays différents.



Figure 1: Localisations des sites de production HagerGroup dans le monde

1.2.2 Site principale



Figure 2: Localisation du plus gros site Hager à Obernai

1.3 Secteurs d'activité

Nos marques Les spécialistes

:hager

Hager dispose d'une offre complète de produits et systèmes pour la distribution électrique dans les bâtiments industriels, les locaux professionnels et l'habitat.

**B.
Berker**

Les interrupteurs Berker sont utilisés dans le monde entier pour apporter plus de confort, de simplicité et d'esthétique à votre quotidien.

ELCOM.

Elcom est le spécialiste des systèmes modernes d'interphonie et des solutions de communication personnalisées.

E3/DC
ENERGY STORAGE

E3/DC fabrique en Allemagne et en Suisse des solutions innovantes dans le stockage de l'énergie et l'électromobilité.

BOCCHIOTTI
IBOCO

Bocchiotti et Iboco sont spécialistes dans la fabrication et la commercialisation de solutions et de services dans le domaine des chemins de câbles et des petits coffrets de distribution pour le segment résidentiel et l'industrie.

eficia
smart building®

Eficia est le spécialiste de la performance énergétique des bâtiments avec une solution innovante et exclusive pilotée 24/7.

Pmflex

Pmflex est le spécialiste et leader du marché des systèmes de conduits flexibles et rigides pour les installations basse tension en Europe.

Figure 3: Les différentes marques HagerGroup

1.4 Produits phares

Disjoncteur magnéto-thermique



Interrupteur différentiel



Interrupteur



Prises électriques



Borne de recharge



Tableau de distribution



Figure 4: Produits phares de HagerGroup

- Disjoncteur magnéto-thermique : disjoncteur protégeant contre les courts-circuits (magnéto) et contre les surcharges (thermique).
- Interrupteur différentiel : protège les personnes en détectant une fuite de courant à la terre.
- Borne de recharge : permet la recharge rapide de véhicules électriques.
- Tableau de distribution : tableau hébergeant les différents éléments constituant une installation électrique (ex: disjoncteurs, interrupteurs différentiels...).

1.5 Chiffres clés

Hager Group est une entreprise de grande envergure avec plus de 13000 collaborateurs, un chiffre d'affaire de plus de 3 milliards d'euros et 22 sites de production répartis en Europe et en Asie.

3.2

milliards d'euros de
chiffre d'affaires

13,000

collaborateurs

>100

pays où les solutions
Hager Group sont distribuées

22

sites de production

Figure 5: Chiffres clés de l'année 2023

- Chiffre d'affaire : chiffre d'affaire total du groupe (Europe, Asie, USA).
- Collaborateurs : différents services au sein de HagerGroup (RH, vente, informatique, maintenance...).
- Principaux sites de production : Obernai, Blieskastel, Bieruń.

1.6 Organisation des services informatiques

Le service informatique (SI)¹ au sein de HagerGroup se divise en quatre grands groupes (cf. Figure 6) :

1. une équipe en charge de l'administration des réseaux industriels (IIT²),
2. une équipe en charge du développement d'applications pour le groupe,
3. une équipe en charge de l'administration des réseaux bureautique,
4. les relais IIT en charge de la maintenance des équipements informatiques industriels.

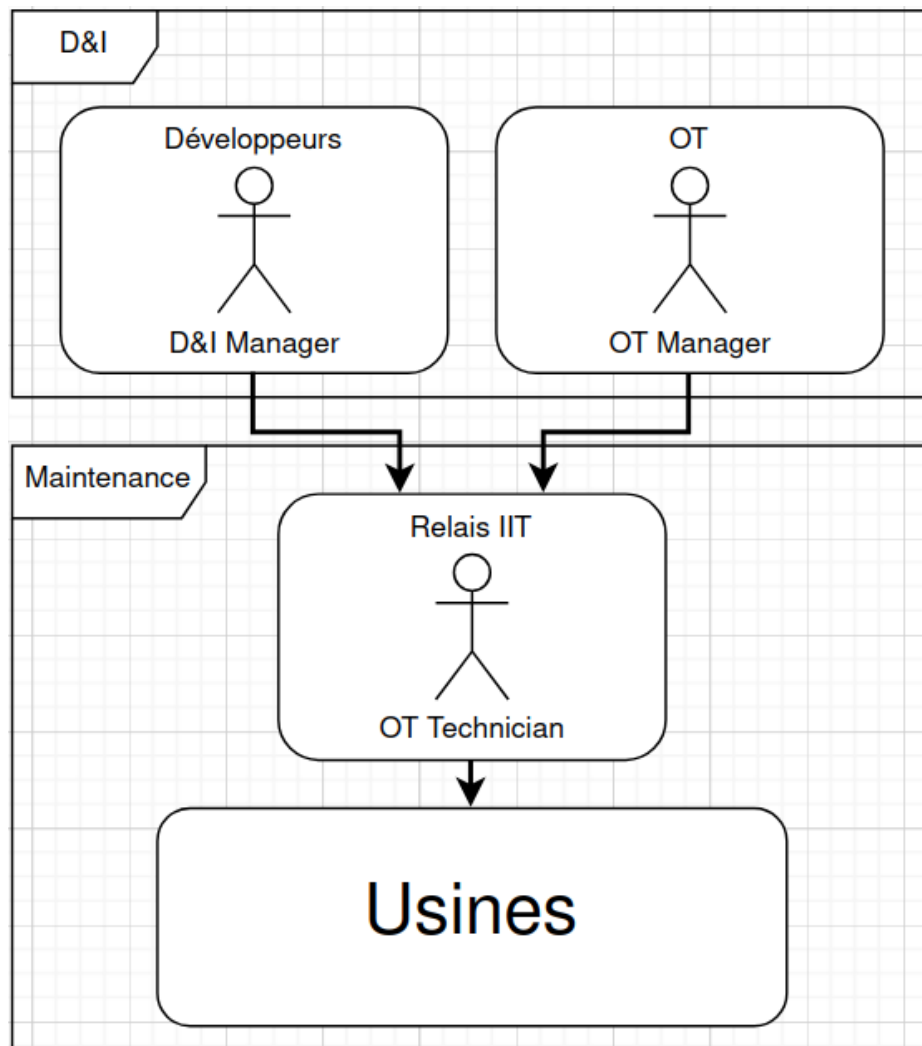


Figure 6: Schéma de l'organisation du SI

¹Service Informatique

²Industrial Information Technology, synonyme : OT

1.7 Présentation de l'équipe

Les relais IIT sont l'interface entre l'équipe IIT (Industrial Information Technology) et la production. Ils sont en charge de la maintenance des équipements informatiques présents au sein des usines et sont donc obligés de travailler avec des automaticiens, des techniciens de maintenance, avec l'équipe en charge des serveurs ainsi qu'avec les développeurs.

Ci-dessous une liste non exhaustive des collaborateurs de l'apprenti :

Nom	Rôle	Description
Nicolas	Relais IIT	Nicolas est en charge de la maintenance des PC de production du site d'Obernai. Il s'assure d'appliquer les politiques de cybersécurité, les sauvegardes et il cartographie le réseau.
Estelle	Automaticienne	Estelle est en charge des automates de l'usine 2 du site d'Obernai et maintient également le parc informatique de cette usine.
Quentin	Ingénieur Système et Réseau	Quentin est chargé des serveurs Veeam ainsi que de l'architecture et de la mise en place de nouvelles infrastructures réseaux.
Sacha	Ingénieur Système	Sacha gère la connectivité machine de toutes les usines HagerGroup.
Patrick	Technicien de maintenance	Patrick gère la maintenance de plusieurs lignes de production et maintient également le parc informatique.
Thomas	Chef de la maintenance	Thomas est le chef de la maintenance de l'usine 1 et aide au maintien du matériel informatique de l'usine dont il est en charge.

Agent Veeam

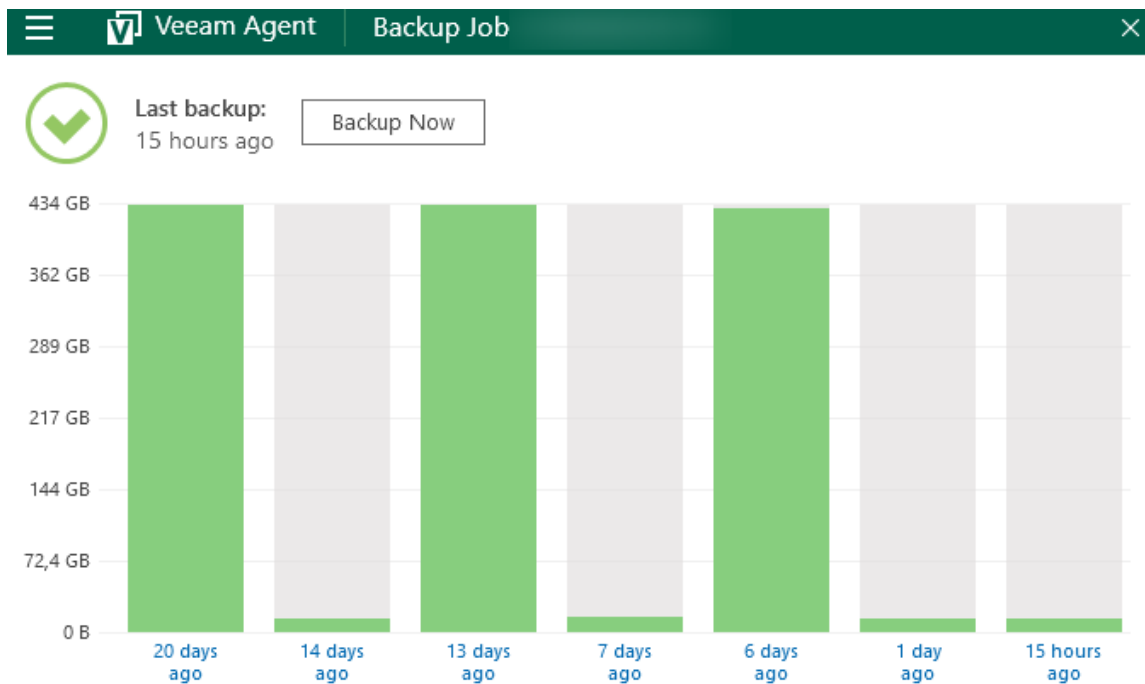


Figure 8: Historique des sauvegardes d'un PC réalisées par l'agent Veeam

Cette sauvegarde hebdomadaire est incrémentale, sur ce diagramme une sauvegarde sur deux est incrémentale.

2.2 Formation des utilisateurs

Formation des utilisateurs aux technologies en place au sein de l'entreprise :

- utilisation de la console Ivanti pour la gestion du parc informatique,
- configuration des sauvegardes Veeam,
- utilisation du serveur SFTP pour le transfert sécurisé de fichiers par le réseau.

2.3 Cartographie du réseau industriel

La cartographie inclut les PC de production, les PLC³ ainsi que les périphériques IP⁴ tels que des caméras, imprimantes, IHM⁵... (cf. Figure 9). Cette cartographie est utilisée pour sécuriser le parc informatique et permet également de définir un niveau de criticité pour chaque poste (BCP⁶ code). Ce code permet de déterminer l'impact de l'arrêt d'un PC sur la production, qu'il soit dû à une panne ou une cyberattaque.

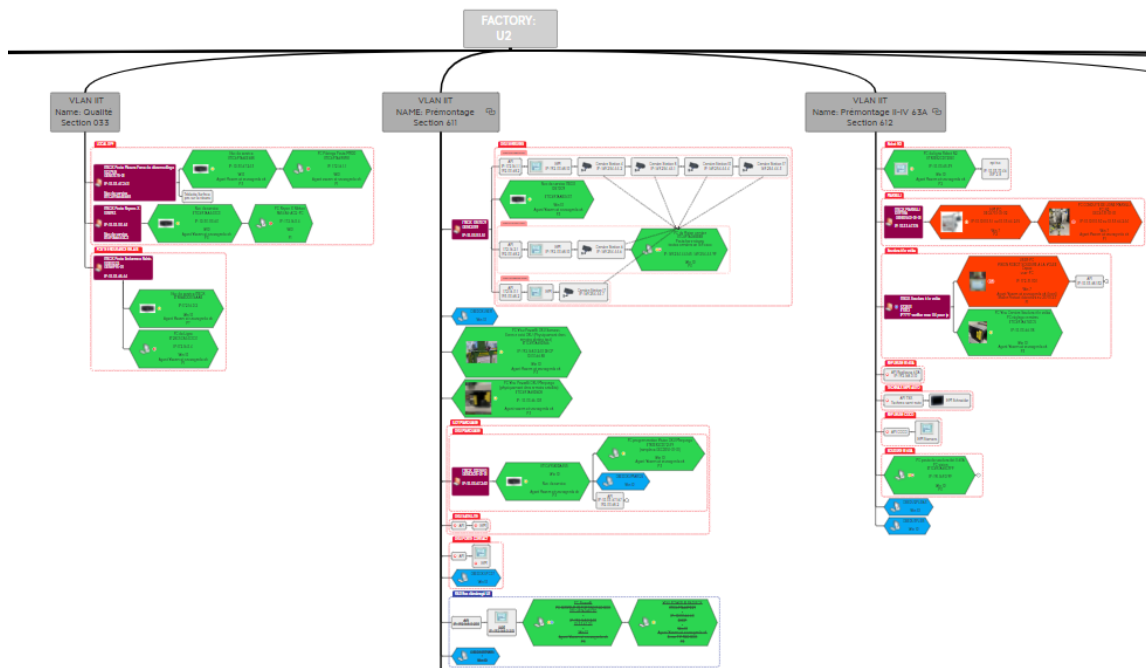
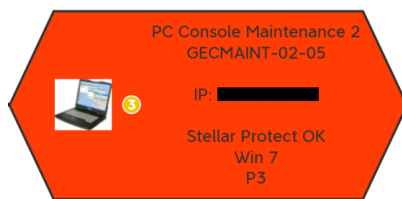
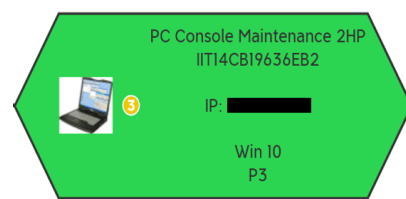


Figure 9: Cartographie du réseau XMind



(a) PC extrait de la cartographie



(b) PC extrait de la cartographie

Figure 10: Extrait de la cartographie

³Programmable Logic Controller

⁴Internet Protocol

⁵Interface Homme Machine

⁶Business Continuity Plan

- BCP_P1 : arrêt de la production en cas d'arrêt du PC
- BCP_P2 : fonctionnement en marche dégradée en cas d'arrêt du PC
- BCP_P3 : aucun impact sur la production en cas d'arrêt du PC

2.4 Provisioning de postes

Les nouveaux PC sont provisionnés par PXE⁷, enrôlés dans l'AD⁸ à l'aide du compte intune et leurs informations renseignées dans la console Ivanti à l'aide de l'outil Custom Data Form (cf. Figure 11).

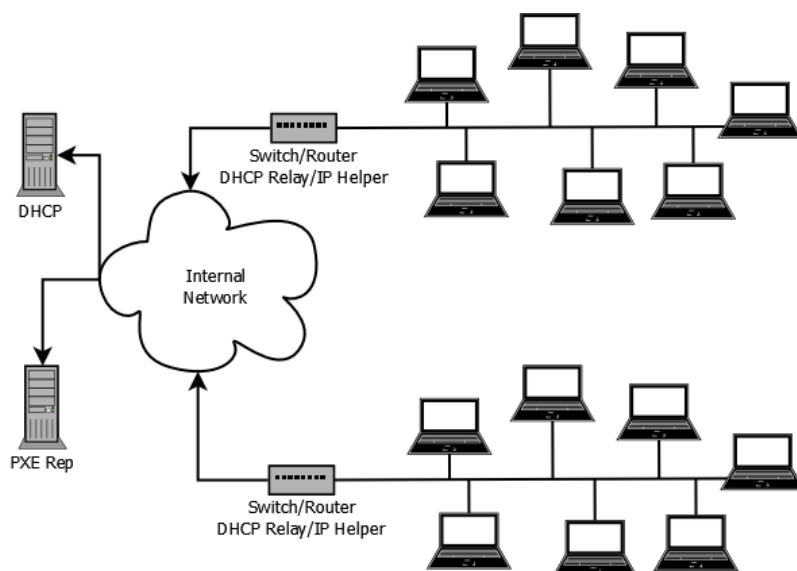


Figure 11: Provisioning de nouveaux PC via PXE et Ivanti.

2.5 Migration de PC de production

Les anciennes applications ainsi que leurs configurations sont transférées sur le nouveau PC si possible, sinon une mise à jour est faite vers un applicatif plus récent. Le PC est ensuite testé et validé dans un environnement de test puis mis en production.



Figure 12: Migration de PC de production depuis Windows 7 vers Windows 10

⁷Preboot Execution Environment

⁸Active Directory

2.6 Ecriture de scripts

2.6.1 Script de purge de disque :

```
1 #Requires AutoHotkey v2.0
2
3 ConfigFile := "config.ini"
4
5 Drive := IniRead(ConfigFile, "Settings", "Drive", "")
6
7 Output_file := IniRead(ConfigFile, "Settings", "LogFile", "C:\tmp\log.
   txt")
8
9 Dir_Path := IniRead(ConfigFile, "Settings", "FolderPath", "")
10 FileDeleteDate := IniRead(ConfigFile, "Settings", "FileRemoveTime", "
   180")
11 DiskTreshold := IniRead(ConfigFile, "Settings", "DiskTreshold", "80")
12 TresholdDate := IniRead(ConfigFile, "Settings", "TresholdFileTime", "
   180")
13
14 FileDeleteDate := -FileDeleteDate
15 TresholdDate := -TresholdDate
16
17 Wait := IniRead(ConfigFile, "Settings", "WaitTime", "5") * 60 * 60 *
   1000 ; Converti le temps en ms
18
19 Loop {
20     Current_Date := A_Now
21
22     FreeSpace := Round(DriveGetSpaceFree(Drive) / 1000, 2) ; Obtenir l'
   espace libre en octets.
23     Capacity := Round(DriveGetCapacity(Drive) / 1000, 2)
24     Percentage := Round(FreeSpace * 100 / Capacity)
25
26     if (Percentage > DiskTreshold){
27         Date_new := DateAdd(Current_Date, TresholdDate, "Days")
28     } else {
29         Date_new := DateAdd(Current_Date, FileDeleteDate, "Days")
30     }
31
32     Loop Files Dir_Path "\*.*", "FR"
33     {
34         FileName := A_LoopFileName
35         FilePath := A_LoopFilePath
36         Time := A_LoopFileTimeModified
37         if ( Time < Date_new ) {
38             FileDelete FilePath
39             Line := "Removed File : " FilePath " at " FormatTime(Current_Date
   , "yyyy-MM-dd H:mm") "`n"
40             FileAppend(Line, Output_file)
41         }
42     }
43     Sleep(Wait)
44 }
```

Listing 1: Script de purge de disque

3 Projet E6

3.1 Gestion de projet

L'objectif de ce projet est de pouvoir mesurer la qualité de l'air des usines ainsi que d'envoyer des alertes si des seuils sont dépassés.

Pour cela, un capteur de qualité d'air sera relié à un microcontrôleur qui va se charger de récupérer les données. Il va ensuite s'authentifier et les envoyer au broker MQTT hébergé sur le serveur Linux.

Le serveur Linux va ensuite récupérer les informations, les stocker dans une base de données, les afficher sur une Dashboard et envoyer des alertes si besoin.

3.1.1 Diagramme de contexte :

Le capteur de QAI effectue des mesures à intervalle régulier, ces mesures sont transmises au microcontrôleur ESP32 qui les transmet à son tour au serveur. Le serveur traite les données, les stocke et les affiche sur une Dashboard.

La Dashboard est accessible par les utilisateurs pour visualiser les données et par les administrateurs pour configurer le système.

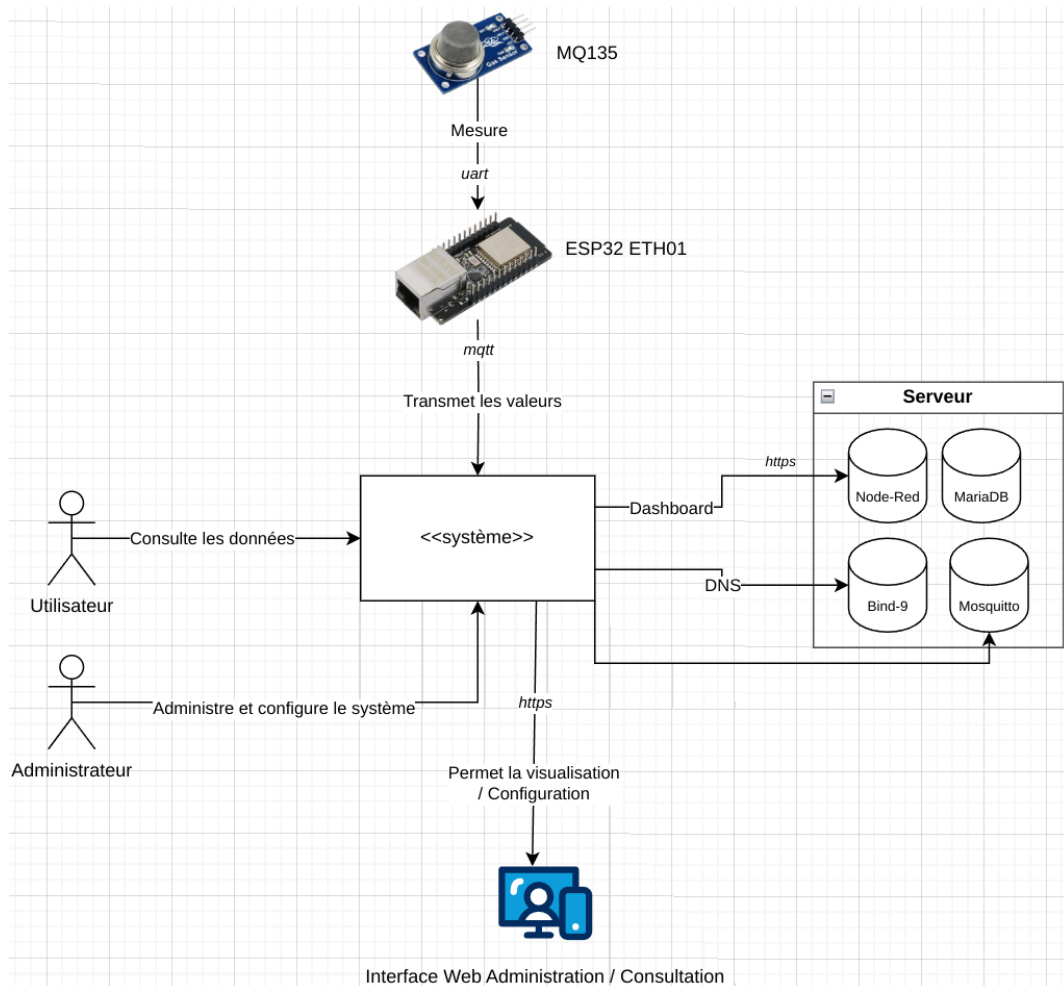


Figure 13: Diagramme de contexte prévisionnel

3.1.2 Diagramme UML de cas d'utilisation :

Le technicien informatique accède à l'interface de paramétrage du système après s'être authentifié.

Il peut paramétrer les alertes et le système de capteurs depuis cette interface. L'utilisateur peut consulter les données après authentification et reçoit des alertes mail si un seuil de QAI est dépassé.

Le microcontrôleur récupère les données des capteurs et les transmet au serveur de centralisation.

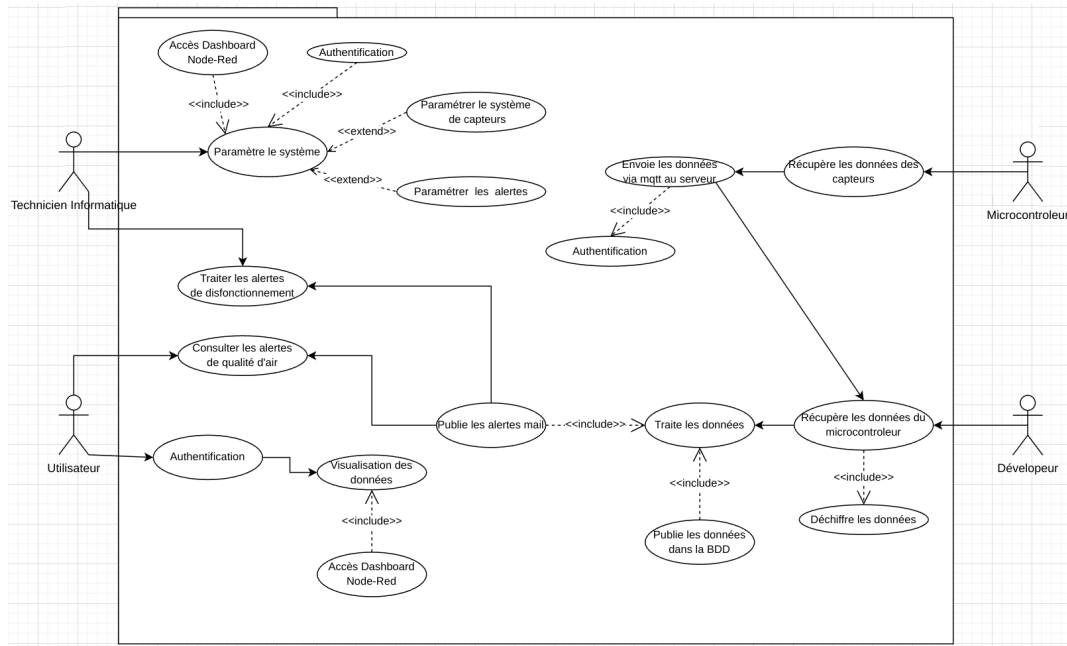


Figure 14: Diagramme des cas d'utilisation prévisionnel

3.1.3 Diagramme de séquence de visualisation des données :

Lorsqu'un client souhaite visualiser les données des capteurs, il va accéder à la Dashboard via un navigateur internet.

L'URL sera convertie en adresse IP par le serveur DNS, puis le serveur Authelia se chargera de l'authentification de l'utilisateur.

Enfin, Node-Red va récupérer les données dans la base de données MariaDB et les afficher sur le poste client.

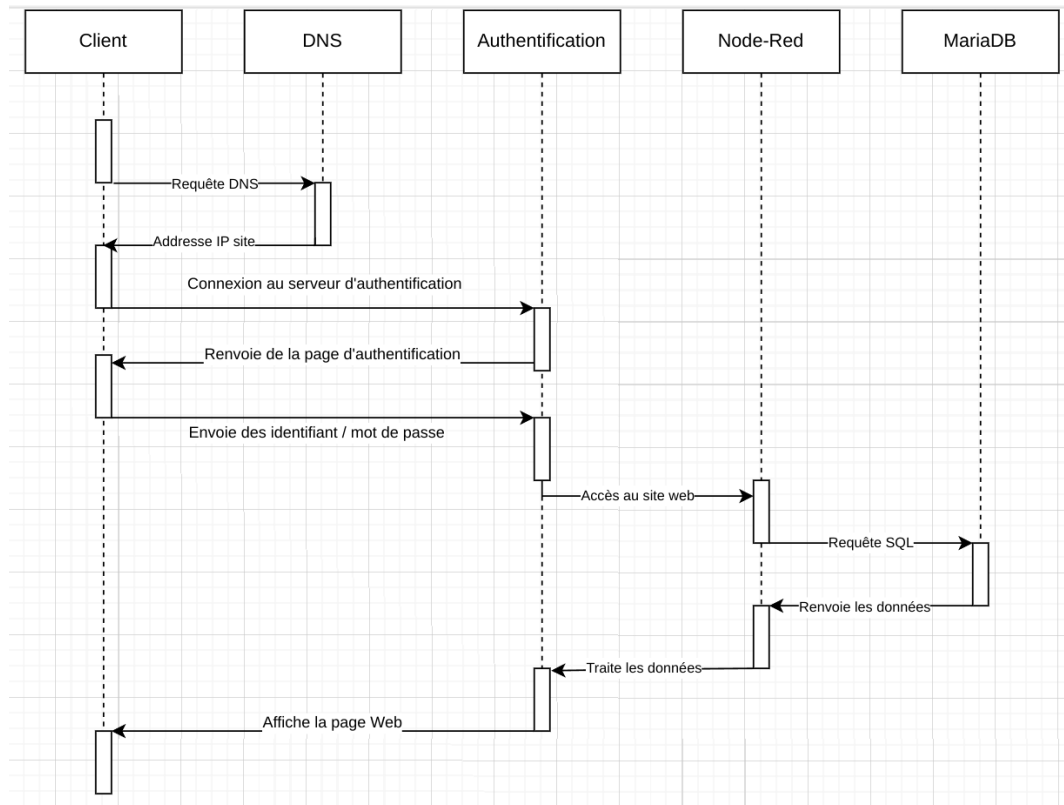


Figure 15: Diagramme de séquence de visualisation des données

3.1.4 Diagramme de séquence de réception / envoi des données :

Le capteur de QAI transmet les données récoltées au microcontrôleur ESP32 qui va ensuite les publier sur le broker MQTT. Le broker MQTT publie les données reçues aux clients qui ont souscrits au topic. Le programme C est un de ces clients et il se charge de récupérer les données et de les écrire dans la base de données.

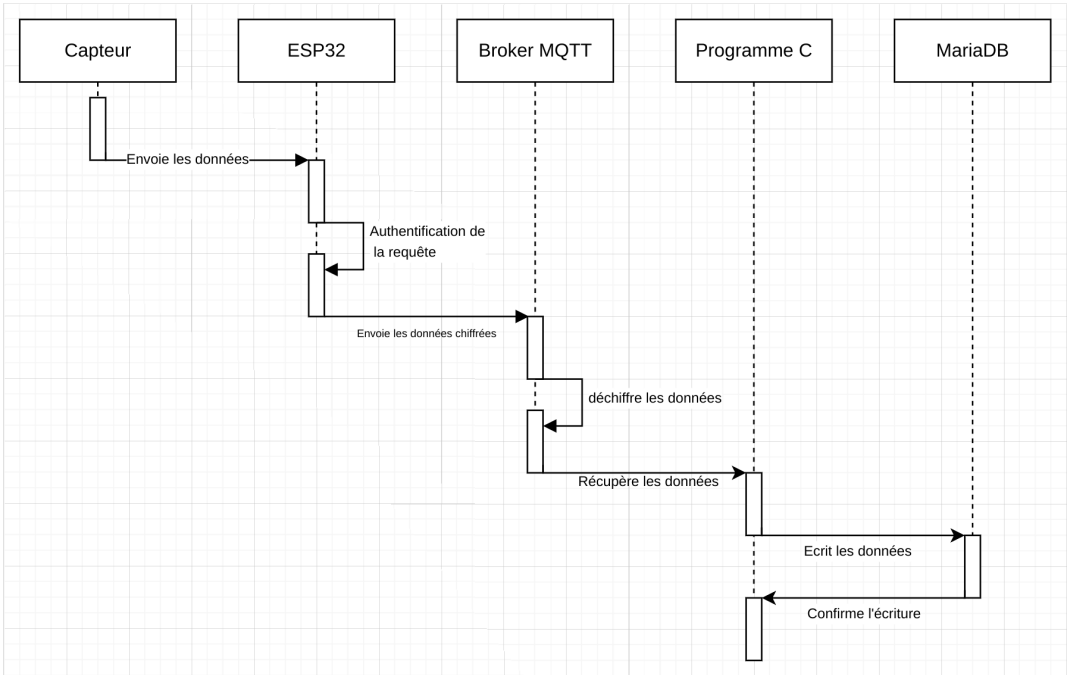


Figure 16: Diagramme de séquence d'envoi des données

3.1.5 Diagramme de Gantt

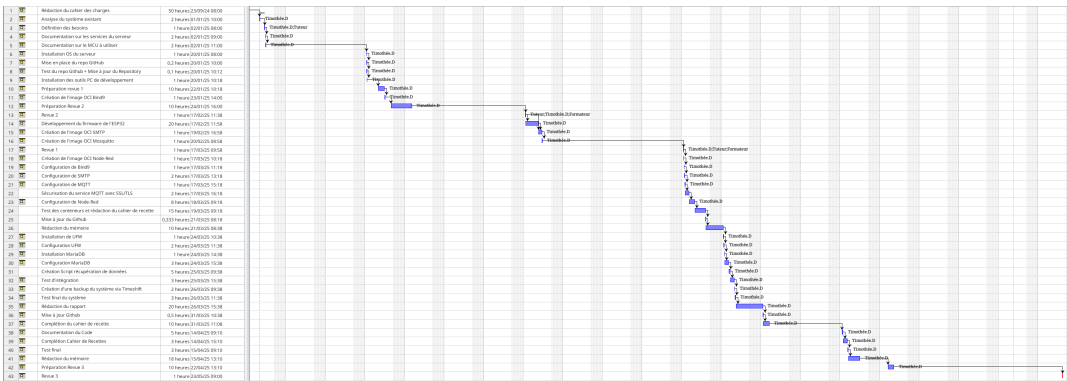


Figure 17: Diagramme de Gantt prévisionnel

3.1.6 Structure interne de la base de données

Cette base de données contient quatre tables : "capteur1", "capteur2", "capteur3" et "capteur4".

Chaque table a les colonnes suivantes :

- **id** : Identifiant unique.
- **nom_capteur** : Nom du capteur.
- **valeur** : Valeur mesurée.
- **timestamp** : Horodatage de la mesure.

Chaque table stocke des données de capteurs différents avec une structure identique.

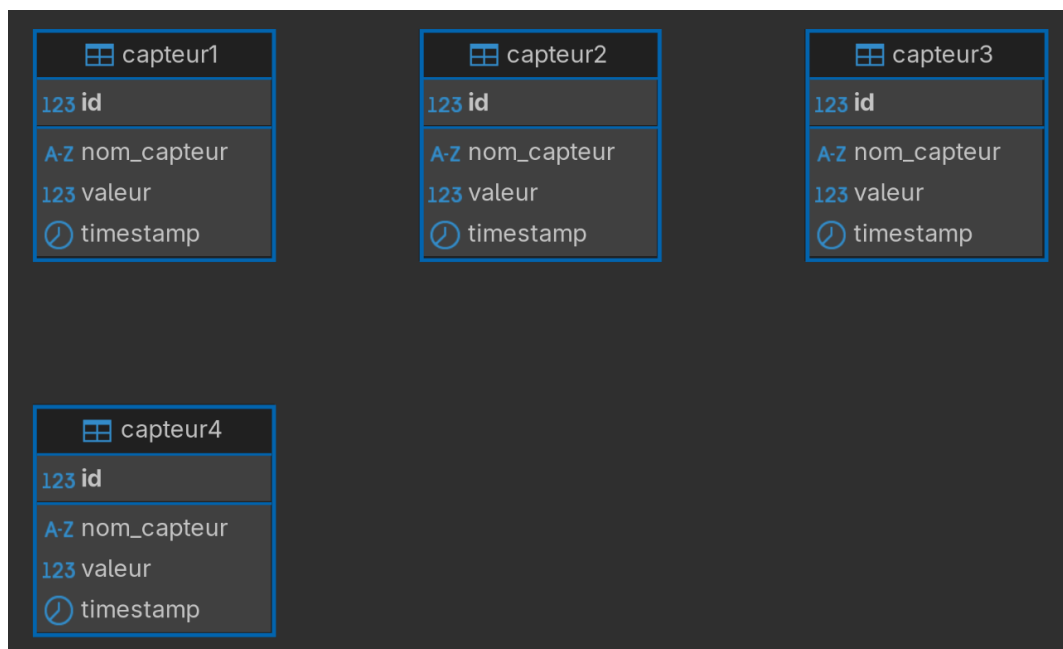


Figure 18: Structure interne de la base de données MariaDB

3.2 Matériels utilisés

- ESP32 WT32-ETH01
- Intel NUC
- Capteur de qualité d'air intérieur MQ135

3.3 Logiciels Utilisés

- C : Programmation de l'ESP32 et des scripts
- Podman : Création des conteneurs OCI
- Node-Red : Création de l'interface de visualisation des données
- SSH : Connexion au serveur à distance
- PostFix : Serveur mail SMTP
- UFW : PareFeux
- MQTT : Protocol de communication utilisé pour transmettre les données entre l'ESP32 et le serveur
- SDS011 : Capteur de qualité d'air
- MariaDB : Base de données
- Debian : Système d'exploitation GNU/Linux
- Sphinx : Logiciel de documentation
- Git + Github : Logiciel et serveur de versionning
- Javascript + CSS

3.4 NodeRed

Node-Red est un environnement de développement basé sur des flux, utilisant une interface glisser-déposer pour créer des automatisations en connectant des nœuds qui représentent des entrées, des traitements et des sorties, le tout s'exécutant sur un runtime Node.js.

Node-Red s'exécute dans un environnement conteneurisé dans une image suivant le format standard OCI. L'image officielle fournie par Node-Red a été utilisée et récupérée via podman.

```
1 podman pull node-red
2 podman run -it -p 1880:1880 --name dashboard --network host dashboard -
  v node_red_data:/data
3 podman start dashboard
```

Listing 2: Podman création & lancement du conteneur Node-Red

3.4.1 Flow 1 :

Le flow 1 est en charge de la connexion à la base de données et du formatage des données pour l'affichage dans l'interface utilisateur

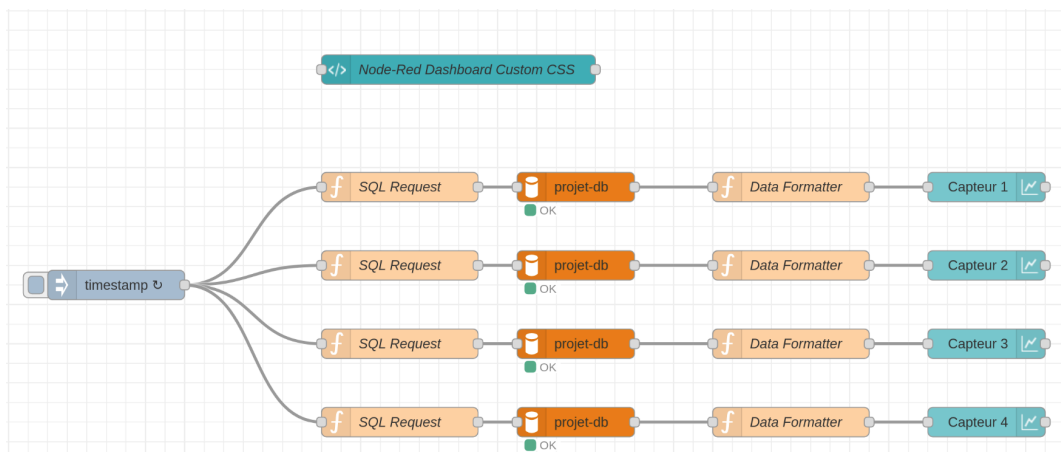


Figure 19: Connexion à la base de données et formatage des données

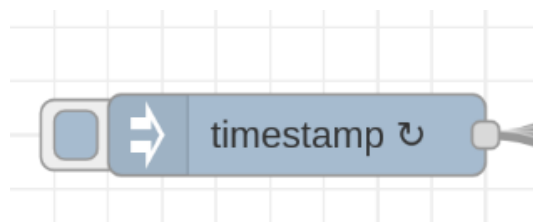


Figure 20: Ce noeud active toutes les minutes les noeuds qui lui sont connectés

3.4.2 Requête SQL de récupération des données

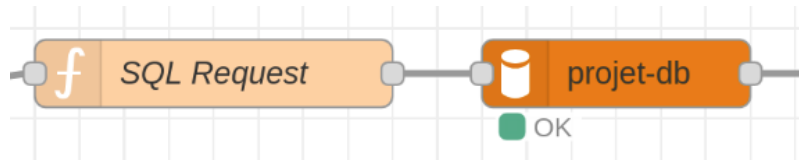


Figure 21: Cette combinaison de noeuds envoie une requête SQL à la base de données pour récupérer les 1000 dernières valeurs d'un capteur

L'image montre deux nœuds dans un flux Node-RED.

Le premier nœud, étiqueté "SQL Request", est utilisé pour exécuter des requêtes SQL sur une base de données.

Ce nœud est configuré pour envoyer une requête SQL à la base de données, récupérer les valeurs et les transmettre sous forme de message `msg.payload` au nœud suivant dans le flux.

Le second nœud, étiqueté "project-db", représente la base de données à laquelle la requête SQL est envoyée (projet-db).

Ce nœud est configuré avec les informations de connexion nécessaires pour accéder à la base de données "project-db" cf Figure 22.

Une fois la requête exécutée, les résultats sont renvoyés au nœud "SQL Request", qui les transmet ensuite au reste du flux.

🏷️ Name	BDD Capteurs
🌐 Host	192.168.253.61
🔄 Port	3306
👤 User	projet
🔒 Password	
🗄️ Database	projet-db

Figure 22: Paramètres du noeud de connexion à la base de données

```

1 SELECT *
2 FROM `projet-db`.capteur1
3 ORDER BY id DESC
4 LIMIT 1000;

```

Listing 3: Requête SQL de récupération des données du capteur 1

Les 1000 dernières données sont récupérées par une requête SQL cf Listing 3, triées par ordre descendant afin d’avoir les données les plus récentes.

3.4.3 Affichage des données

 Group	[Charts] Capteur 1 			
 Size	auto			
 Label	Capteur 1			
 Type	 Line chart 	☐ enlarge points		
X-axis	last	1	hours 	OR 1000 points
X-axis Label	 HH:mm:ss			☐ as UTC

Figure 23: Paramètres du noeud d’affichage des données de la base de données

3.4.4 Code de conversion des données de la base de données pour l’affichage

Les données sont ensuite affichées dans un graphique montrant l’historique des mesures.

```
1 // Récupérer les résultats de la requête MySQL
2 let data = msg.payload;
3
4 // Créer un tableau pour stocker les résultats formatés
5 let formattedData = [];
6
7 // Parcourir chaque ligne de données
8 data.forEach(row => {
9     // Chaque ligne a une valeur et un timestamp
10    let point = {
11        "x": new Date(row.timestamp).getTime(), // Convertir le
12            timestamp en millisecondes
13        "y": row.valeur // La valeur associée
14    };
15
16    // Ajouter le point au tableau formaté
17    formattedData.push(point);
18
19 // Envoyer les données formatées vers le graphique
20 msg.payload = [{
21     "series": ["Valeurs"],
22     "data": [formattedData],
23     "labels": ["Timestamp"]
24 }];
25
26 return msg;
```

Listing 4: Code Javascript de transformation des données de la base de données pour l’affichage dans les graphiques de l’interface utilisateur

Ce code JavaScript, utilisé dans un nœud Node-RED, transforme les résultats d’une requête MySQL pour les adapter à un graphique.

Il extrait les données de ‘msg.payload’ et les reformate en objets avec des timestamps convertis en millisecondes et des valeurs associées.

Ces objets sont stockés dans un tableau ‘formattedData’.

Ensuite, les données formatées sont structurées dans ‘msg.payload’ avec des séries, des données et des labels, puis renvoyées pour être utilisées par un nœud de visualisation graphique.

3.4.5 CSS Custom

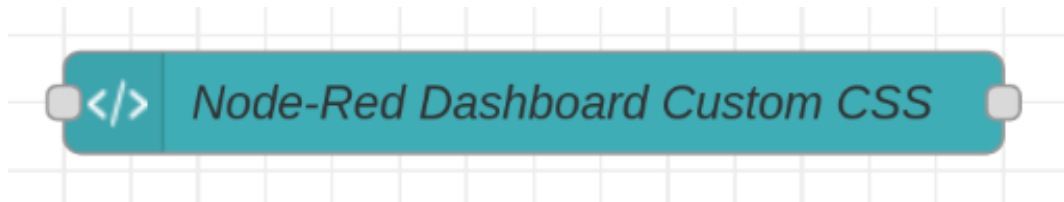


Figure 24: Noeud CSS permettant de changer l'apparence de l'interface utilisateur

```
1 <style>
2 // Fond principal avec un dégradé bleu foncé
3 body {
4     background: -webkit-linear-gradient(
5         55deg,
6         #002B5B 0%, // Bleu foncé
7         #00448A 50%, // Bleu moins foncé
8         #001F3F 100% // Bleu clair
9     );
10 }
11
12 // Barre supérieure
13 body.nr-dashboard-theme md-toolbar,
14 body.nr-dashboard-theme md-content md-card {
15     background-color: transparent !important; // Fond transparent
16     color: #FFFFFF !important; // Texte en blanc
17 }
18
19 // Menu de gauche
20 // Barre latérale
21 body.nr-dashboard-theme md-sidenav {
22     color: white !important; // Texte en blanc
23     background-color: rgba(0,0,0,0) !important; // Fond transparent
24 }
25 // Survol de la sélection
26 a.md-no-style, button.md-no-style {
27     border-radius: 10px !important; // Coins arrondis
28 }
29 // Élément sélectionné
30 .nr-menu-item-active div button {
31     border-right: 4px solid rgb(41, 98, 255) !important; // Bordure
32     // droite bleue
33 }
34 // Liste
35 body.nr-dashboard-theme md-sidenav div.md-list-item-inner {
36     color: white !important; // Texte en blanc
37     background-color: rgba(40,85,165,1) !important; // Fond bleu
38     border-radius: 10px !important; // Coins arrondis
39 }
40 // Groupes
41 ui-card-panel {
42     background-color: rgba(255,255,255,0.1) !important; // Fond
43     // semi-transparent blanc
44     border-radius: 10px !important; // Coins arrondis
```

```

44 }
45 .nr-dashboard-theme ui-card-panel {
46     border: none !important; // Pas de bordure
47 }
48 // Noms des groupes
49 .nr-dashboard-theme ui-card-panel p.nr-dashboard-cardtitle {
50     color: rgba(255, 255, 255, 0.5) !important; // Texte blanc semi
        -transparent
51 }
52
53 // Boutons
54 .nr-dashboard-theme .nr-dashboard-button .md-button {
55     background-color: rgba(255,255,255,.1) !important; // Fond semi
        -transparent blanc
56 }
57 .md-button {
58     border-radius: 10px !important; // Coins arrondis
59 }
60
61 // Menu déroulant
62 .nr-dashboard-theme md-select-menu {
63     background-color: rgba(40,85,165,1) !important; // Fond bleu
64 }
65 .nr-dashboard-theme md-select-menu, .nr-dashboard-theme md-select-
    menu md-option {
66     background-color: transparent !important; // Fond transparent
67 }
68 .nr-dashboard-theme .md-select-menu-container {
69     border-radius: 10px !important; // Coins arrondis
70     border: none !important; // Pas de bordure
71 }
72 .nr-dashboard-theme md-select-menu md-option[selected] {
73     color: white !important; // Texte en blanc
74     background-color: #1a3566 !important; // Fond bleu foncé
75 }
76
77 // Modèle
78 md-card>img, md-card>md-card-header img, md-card md-card-title-
    media img {
79     border-radius: 10px !important; // Coins arrondis
80 }
81
82 // Fond du texte du sélecteur de couleur
83 .nr-dashboard-theme .color-picker-input-wrapper > input {
84     color: white !important; // Texte en blanc
85     background-color: transparent !important; // Fond transparent
86 }
87 </style>

```

Listing 5: Code CSS custom de l'interface utilisateur

3.4.6 Flow 2 :

Le flow 2 est en charge de la souscription au broker MQTT et de l'affichage en temps réel des données dans l'interface utilisateur.

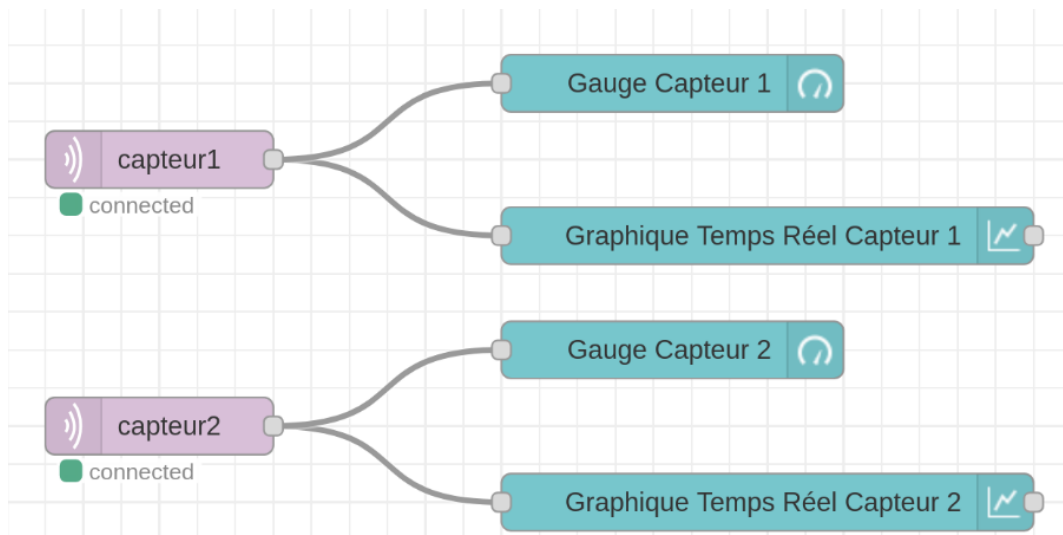


Figure 25: Affichage en temps réel des données

3.4.7 Authentification :

L'accès à la page web d'administration est protégé par mot de passe permettant de sécuriser l'accès.

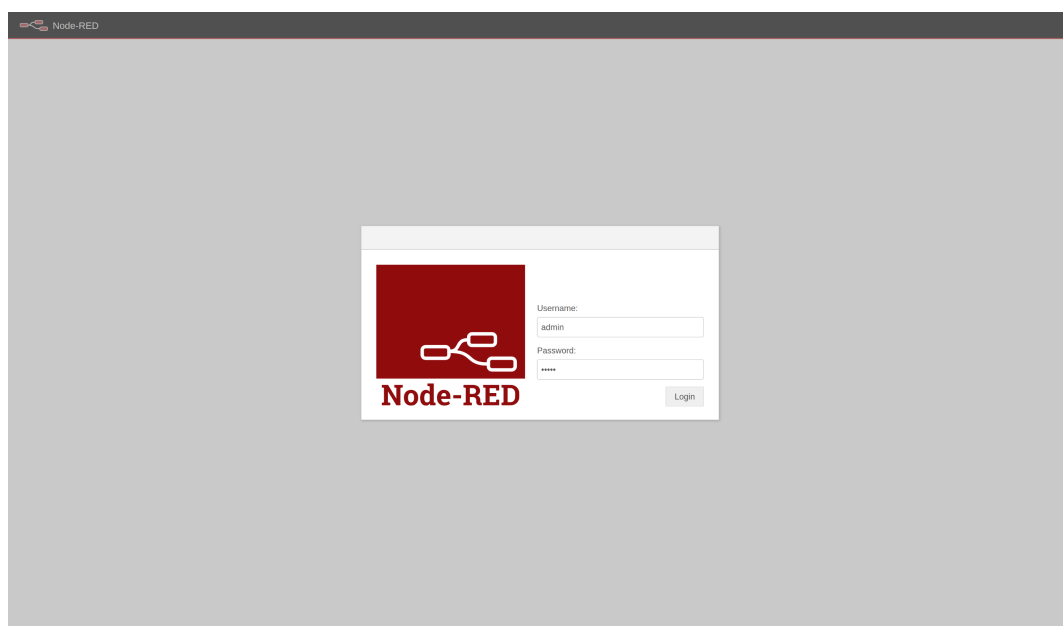


Figure 26: Accès à la console d'administration et de visualisation protégé par mot de passe

3.4.8 Interface de visualisation des données :

Les données stockées dans la base de données sont accessibles depuis l'onglet Charts.

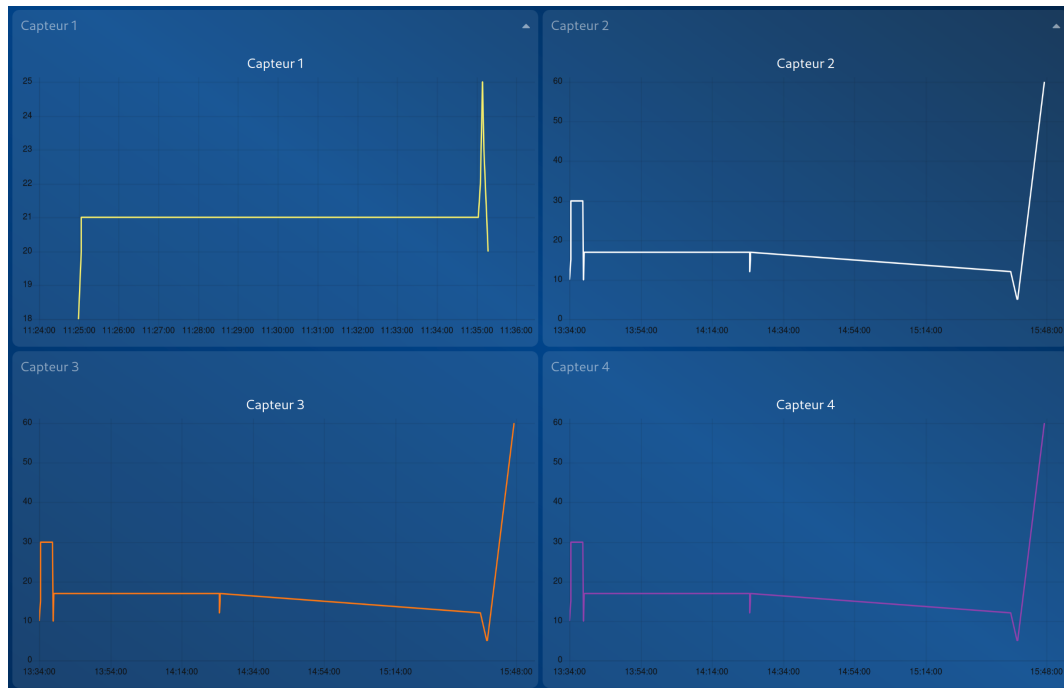


Figure 27: Interface utilisateur de visualisation de l'Histoire des données des capteurs

Les données en temps réel sont récupérées depuis mosquitto en souscrivant au topic correspondant au capteur et affiche les données dans un graphique.

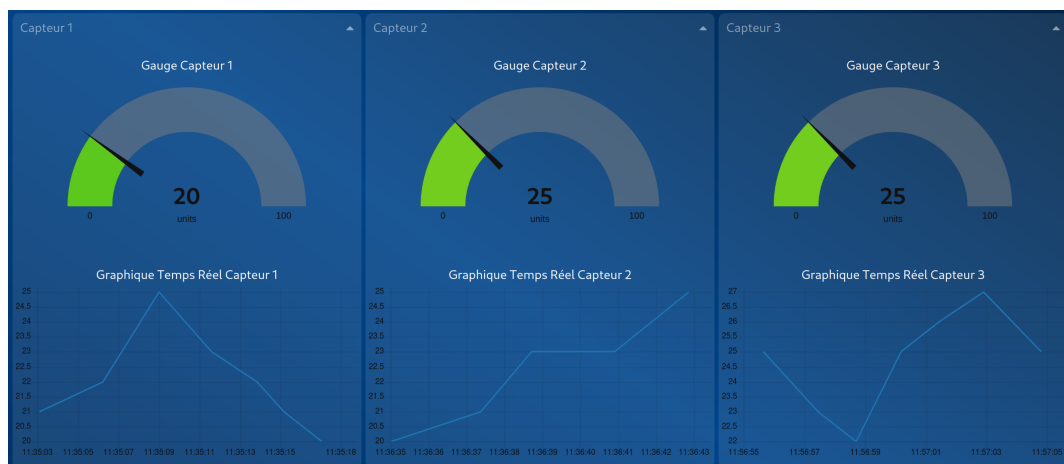


Figure 28: Interface utilisateur de visualisation des données en temps réel

3.5 Script de transfert des données depuis MQTT vers MariaDB

Ce script est en charge du transfert des données depuis MQTT vers la base de données MariaDB.

Une instance du script est lancée par capteur.

Ce script fonctionne sur une base d'interruptions, chaque fois que le broker MQTT publie une donnée une interruption est générée et la donnée est envoyé dans la base.

```
1 if(argc < 8){ // Usage
2     printf("Usage: ./mqtt_to_bdd.elf hôte port topic utilisateur_sql
3         mdp_bdd base_sql nom_capteur\n");
4     return 1;
5 }
```

Listing 6: Vérification de la bonne utilisation du script

```
1 conn = mysql_init(NULL); // Initialise l'instance mariadb
2
3 if (conn == NULL) {
4     fprintf(stderr, "Erreur d'initialisation de MySQL: %s\n",
5         mysql_error(conn));
6     exit(1);
7 } else printf("SQL instance initiated\n");
8
9 if (mysql_real_connect(conn, "127.0.0.1", argv[4] , argv[5], argv[6],
10     3306, NULL, 0) == NULL) { // Connecte à la base de données
11     fprintf(stderr, "Erreur de connexion à la base de données: %s\n",
12         mysql_error(conn));
13     mysql_close(conn);
14     exit(1);
15 }else printf("Connection to the database succesful\n");
```

Listing 7: Initialisation de l'instance MariaDB et connexion à la BDD

```
1 if(mosquitto_lib_init() != MOSQ_ERR_SUCCESS){ // Alloue les ressources
2     requises par la librairie MQTT
3     printf("Error while init lib\n");
4     exit(1);
5 }
6
7 void *user_data = NULL;
8 struct mosquitto *mqtt_handle = mosquitto_new(argv[3], 0, user_data);
9 // Crée une nouvelle instance MQTT
10 if(mqtt_handle == NULL){
11     printf("Error while creating new instance of mosquitto\n");
12     cleanup(conn, mqtt_handle);
13     exit(1);
14 }
```

Listing 8: Allocation des ressources MQTT et création de l'instance

```

1 if(mosquitto_connect(mqtt_handle, argv[1], atoi(argv[2]), 10) !=
  MOSQ_ERR_SUCCESS){ // Connecte au broker MQTT
2   printf("Error while connecting to mqtt broker\n");
3   cleanup(conn, mqtt_handle);
4   exit(1);
5 } else printf("Connection to the broker successful\n");
6
7 int err = 0;
8 while((err = mosquitto_subscribe(mqtt_handle, NULL, argv[3], 1) !=
  MOSQ_ERR_SUCCESS)){ // Souscrit au topic passé en argument du
  programme
9   printf("Error while subscribing :%d\nRetrying...", err);
10  sleep(1);
11 }
12 printf("Subscribing to topic %s succesful\n", argv[3]);
13
14 err = mosquitto_loop_forever(mqtt_handle, -1, 1);
15 if(err != MOSQ_ERR_SUCCESS){
16   printf("Error while creating the mosquitto loop : %d\n", err);
17   cleanup(conn, mqtt_handle);
18   exit(1);
19 }

```

Listing 9: Connexion au broker et souscription au topic

```

1 void mqtt_callback(struct mosquitto *mqtt, void *user_data, const
  struct mosquitto_message *message){
2   if (message->payloadlen) {
3     // Log sur le terminal des données reçues
4     printf("Message reçu %s: %s\n", message->topic, (char *)message
      ->payload);
5     char query[512];
6     // Insertion des données dans la base de données
7     snprintf(query, 256, "INSERT INTO %s (valeur, nom_capteur)
      VALUES ('%s', '%s')", arg, (char *)message->payload, (char
      *)message->topic);
8     // Log de la Query dans le terminal
9     printf("Query: %s\n", query);
10    if (mysql_query(conn, query)) {
11      fprintf(stderr, "Erreur lors de l'exécution de la requête:
      %s\n", mysql_error(conn));
12    }
13  } else {
14    // Si le message est vide alors log dans le terminal
15    printf("Message reçu %s (message vide)\n", message->topic);
16  }
17 }

```

Listing 10: MQTT Callback

```

1 mosquitto_message_callback_set(mqtt_handle, mqtt_callback);

```

Listing 11: Enregistrement de la fonction callback

3.6 MariaDB

Chaque capteur a sa table associée cf Figure 29 contenant un identifiant unique (id), le nom du capteur, les valeurs mesurées ainsi qu'un timestamp (horodatage) permettant de situer la mesure dans le temps cf Figure 30.

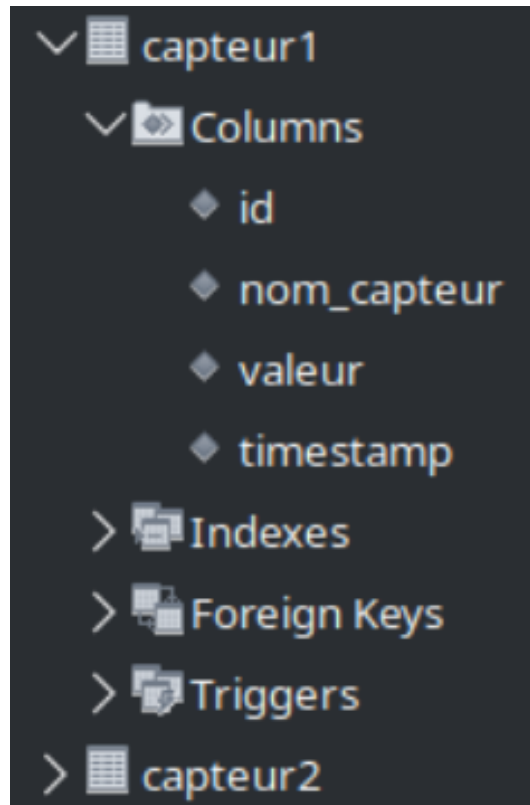


Figure 29: Tables contenant les données des capteurs

id	nom_capteur	valeur	timestamp
1	capteur2	10	2025-04-09 11:34:05
2	capteur2	15	2025-04-09 11:34:21
3	capteur2	20	2025-04-09 11:34:23
4	capteur2	30	2025-04-09 11:34:25
5	capteur2	30	2025-04-09 11:37:50
6	capteur2	10	2025-04-09 11:37:56
7	capteur2	17	2025-04-09 11:38:07
8	capteur2	17	2025-04-09 12:24:41
9	capteur2	12	2025-04-09 12:24:43
10	capteur1	15	2025-04-09 12:24:48
11	capteur1	17	2025-04-09 12:24:50
12	capteur3	12	2025-04-09 13:37:52
13	capteur4	5	2025-04-09 13:39:38
14	capteur4	5	2025-04-09 13:39:46
15	capteur2	60	2025-04-09 13:47:20

Figure 30: Tables contenant les données des capteurs

3.6.1 Requête SQL de création de la table d'un capteur

```
1 CREATE TABLE `projet-db`.`capteur1` (  
2   `id` INT NOT NULL AUTO_INCREMENT,  
3   `nom_capteur` VARCHAR(45) NOT NULL,  
4   `valeur` DECIMAL(10, 2),  
5   `timestamp` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
6   PRIMARY KEY (`id`)  
7 );
```

Listing 12: Requête SQL de création de la table du capteur 1

Cette requête SQL crée une table nommée `capteur1` dans la base de données `projet-db`.

La table est conçue pour stocker des informations relatives à un capteur.

Elle contient quatre colonnes : `id`, `nom_capteur`, `valeur`, et `timestamp`.

La colonne `id` est définie comme une clé primaire avec auto-incrémentation, assurant un identifiant unique pour chaque entrée.

`nom_capteur` est une chaîne de caractères de 45 caractères maximum et ne peut pas être nulle, représentant le nom du capteur.

`valeur` est un nombre décimal avec une précision de 10 chiffres, dont 2 décimales, pour stocker les mesures du capteur.

Enfin, `timestamp` enregistre automatiquement la date et l'heure de création de chaque entrée, utilisant par défaut l'heure actuelle.

Cette structure permet de suivre et d'enregistrer efficacement les données des capteurs au fil du temps.

3.7 Broker MQTT

3.7.1 Récupération de l'image Eclipse

Le conteneur OCI du broker MQTT est basé sur l'image officielle de Eclipse avec une configuration personnalisée.

```
1 podman pull docker.io/eclipse-mosquitto
```

Listing 13: Récupération du conteneur officiel eclipse-mosquitto

3.7.2 Premier lancement du conteneur

Le conteneur est lancé une première fois avec une redirection des ports 1883 et 9001 vers l'hôte. Ici, mqtt est le nom de l'image après modification de la configuration.

```
1 podman run -it --name mosquitto-projet -p 1883:1883 -p 9001:9001 mqtt -  
  config mosquitto -c /mosquitto/config/mosquitto.conf
```

Listing 14: Premier lancement du conteneur mqtt avec les bons paramètres

3.7.3 Lancement du conteneur après arrêt

Pour lancer le conteneur après un arrêt.

```
1 podman start mosquitto-projet
```

Listing 15: Lancement du conteneur après un arrêt

3.7.4 Configuration Broker MQTT

```
1 listener 1883  
2 listener 9001  
3 protocol websockets
```

Listing 16: Configuration du broker MQTT

3.8 Lancement automatisé des conteneurs et scripts

Les conteneurs et scripts utilisés dans ce système sont lancés de manière automatisée par un service managé par systemctl. Cela permet une automatisation du lancement et une journalisation des événements.

3.8.1 Service de lancement des conteneurs

```
1 [Unit]
2 Description=Script de lancement des conteneurs
3 After=network.target
4 [Service]
5 ExecStart=/home/projet/scripts/start_scripts.sh
6 Restart=always
7 [Install]
8 WantedBy=default.target
```

Listing 17: Fichier de configuration du service systemctl du lancement des conteneurs

3.8.2 Script de lancement des conteneurs

```
1 #!/bin/bash
2 sleep 10
3 podman start mariadb
4 sleep 5
5 podman start mqtt
6 sleep 5
7 podman start dashboard
8 while true
9 do
10     sleep 100
11 done
```

Listing 18: Script de lancement des conteneurs

3.8.3 Service de lancement des scripts de transfert de données

```
1 [Unit]
2 Description=Service de lancement des scripts de transfert des données
   depuis MQTT vers la base de données MariaDB
3 After=start_script.service
4 [Service]
5 ExecStart=/home/projet/scripts/main.elf localhost 1883 capteur1 projet
   projet projet-db capteur1
6 Restart=always
7 [Install]
8 WantedBy=default.target
```

Listing 19: Service de lancement du script de transfert des données

3.9 Firmware ESP32

3.9.1 Matériel Utilisé :

Microcontrôleur utilisé : ESP32

Carte de développement ESP32 WT32-ETH01

Convertisseur UART vers USB: CH340G

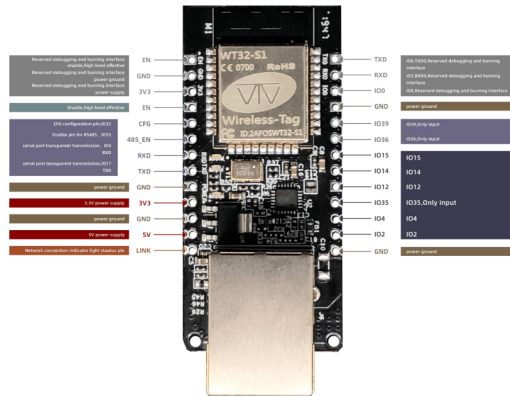


Figure 31: ESP32 WT32-ETH01

3.9.2 Activation de la puce PHY

```
1 gpio_reset_pin(16); // Reset l'état du pin enable de puce PHY
2 gpio_set_direction(16, GPIO_MODE_OUTPUT);
3
4 gpio_set_level(16, 1); // Applique une tension de 3.3v sur le pin
   Enable de la puce PHY pour activer l'horloge de 50MHz
```

Listing 20: Activation de la puce PHY

La puce PHY (Physical Layer) est un composant essentiel dans les systèmes de communication numérique, servant d'interface entre les couches physiques d'un réseau et les couches de liaison de données.

Elle est responsable de la conversion des données numériques en signaux physiques qui peuvent être transmis sur un support de communication, tel qu'un câble ou une onde radio, et inversement.

La puce PHY gère également des fonctions telles que la modulation, la démodulation, la synchronisation et la correction d'erreurs, assurant ainsi une transmission fiable et efficace des données.

3.9.3 Configuration de la MAC

```
1  eth_esp32_emac_config_t emac_config = ETH_ESP32_EMAC_DEFAULT_CONFIG()  
    ; // Configuration ethernet par défaut  
2  emac_config.clock_config.rmii.clock_mode = EMAC_CLK_EXT_IN; //  
    Utilise l'horloge fournie par la puce PHY  
3  emac_config.clock_config.rmii.clock_gpio = EMAC_CLK_IN_GPIO;  
4  
5  eth_mac_config_t mac_conf = ETH_MAC_DEFAULT_CONFIG(); //  
    Configuration MAC par défaut  
6  mac_conf.sw_reset_timeout_ms = 1000;  
7  
8  esp_eth_mac_t *mac = esp_eth_mac_new_esp32(&emac_config, &mac_conf);  
    // Nouvelle instance MAC  
9  ESP_LOGI("MAC :", "esp_eth_mac_new_esp32 done"); // Log  
10 eth_phy_config_t phy_config = ETH_PHY_DEFAULT_CONFIG(); //  
    Configuration de la puce PHY par défaut  
11 phy_config.phy_addr = 1; // Adresse I2C de la puce PHY  
12 phy_config.reset_gpio_num = -1; // Pin reset de la puce PHY non relié  
13  
14 esp_eth_phy_t *phy = esp_eth_phy_new_lan87xx(&phy_config); //  
    Register type of PHY Chip  
15 if (phy != NULL){  
16     ESP_LOGI("PHY :", "esp_eth_phy_new_lan87xx done");  
17 } else {  
18     ESP_LOGE("PHY :", "esp_mac_phy_error");  
19 }  
20 esp_eth_config_t eth_config = ETH_DEFAULT_CONFIG(mac, phy); //  
    Default ETH Config
```

Listing 21: Activation de la puce PHY

Ce code configure la couche MAC (Media Access Control) d'un ESP32 pour une interface Ethernet.

Il commence par définir une configuration par défaut pour l'EMAC (Ethernet MAC) de l'ESP32, en spécifiant l'utilisation d'une horloge externe pour le mode RMII (Reduced Media Independent Interface) et en assignant un GPIO pour l'horloge.

Ensuite, il initialise la configuration MAC avec des paramètres par défaut et un temps de réinitialisation logicielle de 1000 ms.

La fonction `esp_eth_mac_new_esp32` est utilisée pour créer une instance de la couche MAC avec ses configurations.

Le code enregistre également une configuration par défaut pour la puce PHY (Physical Layer), en définissant son adresse I2C et en désactivant la réinitialisation matérielle.

Enfin, il crée une nouvelle instance de la configuration Ethernet en combinant les configurations MAC et PHY, permettant ainsi à l'ESP32 de communiquer via Ethernet.

3.9.4 Installation du driver Ethernet

```
1 ESP_ERROR_CHECK(esp_eth_driver_install(&eth_config, &eth_handle)); //  
   Install the Ethernet Driver  
2 ESP_LOGI("Driver :", "esp_eth_driver_install done");
```

Listing 22: Installation du driver ethernet

Ce code utilise la fonction `esp_eth_driver_install` pour installer et initialiser le pilote Ethernet sur un ESP32, en utilisant la configuration Ethernet définie précédemment.

La fonction `ESP_ERROR_CHECK` est utilisée pour vérifier que l'installation s'est déroulée sans erreur.

Une fois l'installation réussie, un message de log est enregistré pour indiquer que le pilote Ethernet a été correctement installé.

Cela permet de s'assurer que le périphérique est prêt à gérer les communications Ethernet.

3.9.5 Démarrage de la carte Ethernet

```
1 ESP_ERROR_CHECK(esp_eth_start(eth_handle)); // Start Ethernet interface
```

Listing 23: Démarrage de la carte Ethernet

3.9.6 Event Handler du driver Ethernet

```
1  static void eth_event_handler(void *arg, esp_event_base_t event_base,
2                                int32_t event_id, void *event_data)
3  {
4      uint8_t mac_addr[6] = {0};
5
6      switch (event_id) {
7      case ETHERNET_EVENT_CONNECTED:
8          esp_eth_ioctl(eth_handle, ETH_CMD_G_MAC_ADDR, mac_addr);
9          ESP_LOGI(TAG, "Ethernet Link Up");
10         ESP_LOGI(TAG, "Ethernet HW Addr %02x:%02x:%02x:%02x:%02x:%02x",
11                 mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3],
12                 mac_addr[4], mac_addr[5]);
13         break;
14      case ETHERNET_EVENT_DISCONNECTED:
15          ESP_LOGI(TAG, "Ethernet Link Down");
16          break;
17      case ETHERNET_EVENT_START:
18          ESP_LOGI(TAG, "Ethernet Started");
19          break;
20      case ETHERNET_EVENT_STOP:
21          ESP_LOGI(TAG, "Ethernet Stopped");
22          break;
23      default:
24          break;
25      }
26  }
```

Listing 24: Event Handler du driver Ethernet

Un gestionnaire d'événements Ethernet pour ESP32 utilisant ESP-IDF est une fonction qui gère divers événements liés à la connectivité Ethernet. Dans le code fourni, `eth_event_handler` est appelé chaque fois qu'un événement Ethernet se produit, comme la connexion, la déconnexion, le démarrage ou l'arrêt de l'interface Ethernet.

Par exemple, lorsque l'ESP32 se connecte à un réseau Ethernet (`ETHERNET_EVENT_CONNECTED`), il récupère et affiche l'adresse MAC de l'interface. En cas de déconnexion (`ETHERNET_EVENT_DISCONNECTED`), il enregistre un message indiquant que le lien Ethernet est perdu.

Ce gestionnaire permet de surveiller et de réagir aux changements d'état de la connexion Ethernet, facilitant ainsi la gestion des communications réseau sur l'ESP32.

3.9.7 Configuration de la pile IP Netif

```
1  ESP_ERROR_CHECK(esp_netif_init()); // Initialize Lwip TCP/IP Stack
2  ESP_LOGI("NETIF: ", "Netif initialized");
3
4  esp_netif_config_t netif_cfg = ESP_NETIF_DEFAULT_ETH(); // Netif
   config
5  esp_netif_t *eth_netif = esp_netif_new(&netif_cfg); // Create a new
   TCP/IP stack
6  if(eth_netif != NULL){
7      ESP_LOGI("NETIF:", "Netif new created");
8  } else {
9      ESP_LOGE("NETIF:", "Error while creating Netif");
10 }
```

Listing 25: Configuration de la pile IP Netif

La configuration de la pile netif pour un ESP32 utilisant ESP-IDF commence par l'initialisation de la pile TCP/IP LwIP avec `esp_netif_init()`. Cette étape est cruciale pour préparer le système à gérer les communications réseau.

Ensuite, une configuration par défaut pour l'interface Ethernet est définie avec `ESP_NETIF_DEFAULT_ETH()`, qui est stockée dans une structure `esp_netif_config_t`.

Une nouvelle instance de la pile TCP/IP est alors créée avec `esp_netif_new()`, en utilisant la configuration précédemment définie.

Si la création de cette instance est réussie, un message de confirmation est enregistré, indiquant que la nouvelle interface netif a été créée.

En cas d'échec, un message d'erreur est enregistré, permettant ainsi de diagnostiquer les problèmes potentiels lors de la configuration de la pile réseau.

3.9.8 Liaison du driver Ethernet et de la pile IP

```
1 esp_netif_attach(eth_netif, esp_eth_new_netif_glue(eth_handle)); //
   attach Ethernet driver to TCP/IP stack
```

Listing 26: Liaison du driver Ethernet et de la pile IP

Ce code est utilisé dans le contexte de l'ESP32 pour attacher le pilote Ethernet à la pile TCP/IP, permettant ainsi la communication réseau.

La fonction `esp_netif_attach` est appelée avec deux arguments : `eth_netif`, qui représente l'interface réseau précédemment créée, et `esp_eth_new_netif_glue(eth_handle)`, qui crée une "colle" ou un adaptateur entre le pilote Ethernet et l'interface réseau.

Cette étape est cruciale car elle lie le matériel Ethernet (représenté par `eth_handle`) à la pile réseau logicielle, permettant à l'ESP32 de envoyer et recevoir des données via Ethernet.

En effectuant cette attache, l'ESP32 est prêt à participer à des communications réseau, comme l'envoi et la réception de paquets de données.

3.9.9 Configuration IP de la carte

```
1 ESP_ERROR_CHECK(esp_netif_dhcpc_stop(eth_netif)); // Stop DHCP service
   to use Static IP conf
2
3 esp_netif_ip_info_t ip_info;
4 ip_info.ip.addr      = inet_addr("192.168.1.2"); // IP Configuration of
   the esp32
5 ip_info.gw.addr      = inet_addr("192.168.1.1");
6 ip_info.netmask.addr = inet_addr("255.255.255.0");
7
8 ESP_ERROR_CHECK(esp_netif_set_ip_info(eth_netif, &ip_info));
```

Listing 27: Configuration IP de la carte

Ce code configure une adresse IP statique pour l'ESP32, remplaçant l'utilisation du protocole DHCP pour obtenir automatiquement une adresse IP.

Tout d'abord, le service DHCP est arrêté avec `esp_netif_dhcpc_stop(eth_netif)` pour permettre la configuration manuelle de l'adresse IP.

Ensuite, une structure `esp_netif_ip_info_t` est utilisée pour définir les informations IP.

L'adresse IP de l'ESP32 est définie sur 192.168.1.2, la passerelle par défaut sur 192.168.1.1, et le masque de sous-réseau sur 255.255.255.0.

Ces adresses sont converties du format texte en format binaire à l'aide de `inet_addr`.

Enfin, la configuration IP est appliquée à l'interface réseau avec `esp_netif_set_ip_info(eth_netif, &ip_info)`, ce qui permet à l'ESP32 de communiquer sur le réseau local avec l'adresse IP statique spécifiée.

3.9.10 Event Handler de la pile IP Netif

```
1 static void got_ip_event_handler(void *arg, esp_event_base_t event_base
2     ,
3     int32_t event_id, void *event_data)
4 {
5     ip_event_got_ip_t *event = (ip_event_got_ip_t *) event_data; //
6     // Typecast of event_data to ip_event_got_ip type
7     const esp_netif_ip_info_t *ip_info = &event->ip_info; // Extract IP
8     // info
9     ESP_LOGI(TAG, "Ethernet Got IP Address");
10    ESP_LOGI(TAG, "ETHIP:" IPSTR, IP2STR(&ip_info->ip));
11    ESP_LOGI(TAG, "ETHMASK:" IPSTR, IP2STR(&ip_info->netmask));
12    ESP_LOGI(TAG, "ETHGW:" IPSTR, IP2STR(&ip_info->gw));
13 }
```

Listing 28: Event Handler de la pile IP Netif

Ce code définit un gestionnaire d'événements pour l'ESP32 qui est appelé lorsqu'une adresse IP est attribuée à l'interface réseau.

La fonction `got_ip_event_handler` est déclenchée en réponse à un événement spécifique indiquant que l'ESP32 a obtenu une adresse IP.

À l'intérieur de cette fonction, `event_data` est converti en un pointeur de type `ip_event_got_ip_t`, ce qui permet d'accéder aux détails de l'événement.

Ensuite, les informations IP contenues dans cet événement sont extraites et stockées dans une structure `esp_netif_ip_info_t`.

Bien que le code soit incomplet, il est généralement utilisé pour afficher ou utiliser l'adresse IP attribuée, la passerelle et le masque de sous-réseau, permettant ainsi à l'ESP32 de communiquer sur le réseau avec les paramètres IP appropriés.

3.9.11 Configuration MQTT et démarrage du client

```
1 esp_mqtt_client_config_t mqtt_cfg = {
2     .broker.address.hostname = "192.168.1.1",
3     .broker.address.port = 9001,
4     .broker.address.transport = MQTT_TRANSPORT_OVER_WS,
5     .session.keepalive = 120, // Increase keepalive interval
6     .credentials.username = NULL,
7     .credentials.authentication.password = NULL,
8     .credentials.client_id = "Capteur 1"
9 };
10 esp_mqtt_client_handle_t mqtt_handle = esp_mqtt_client_init(&mqtt_cfg);
11 // Create a handle to mqtt client
12 if(mqtt_handle == NULL){
13     ESP_LOGE("MQTT", "Error when init mqtt");
14 } else {
15     ESP_LOGI("MQTT :", "mqtt init done");
16 }
17 ESP_ERROR_CHECK(esp_mqtt_client_start(mqtt_handle)); // Start the mqtt client
```

Listing 29: Event Handler de la pile IP Netif

Ce code configure un client MQTT pour l'ESP32 afin de se connecter à un broker MQTT.

La structure `esp_mqtt_client_config_t` est utilisée pour définir les paramètres de configuration du client MQTT.

Le broker MQTT est spécifié avec l'adresse IP 192.168.1.1 et le port 9001, utilisant le transport WebSocket (`MQTT_TRANSPORT_OVER_WS`).

L'intervalle de keepalive est défini à 120 secondes pour maintenir la connexion active.

Aucun nom d'utilisateur ni mot de passe n'est configuré, mais un identifiant client, "Capteur 1", est attribué pour identifier le client MQTT.

Ensuite, `esp_mqtt_client_init` est appelé avec cette configuration pour initialiser le client MQTT et obtenir un handle.

Si l'initialisation échoue, un message d'erreur est enregistré ; sinon, un message de confirmation indique que l'initialisation du client MQTT a réussi.

3.9.12 Event Handler MQTT

```
1 esp_mqtt_client_register_event(mqtt_handle, ESP_EVENT_ANY_ID,  
2 mqtt_event_handler, mqtt_handle); // Register mqtt event handler  
3 void mqtt_event_handler(void *handler_args, esp_event_base_t base,  
4 int32_t event_id, void *event_data)  
5 {  
6     ESP_LOGD(TAG, "Event dispatched from event loop base=%s, event_id=%  
7     " PRIi32 "", base, event_id);  
8     esp_mqtt_event_handle_t event = event_data;  
9     switch ((esp_mqtt_event_id_t)event_id) {  
10        case MQTT_EVENT_CONNECTED:  
11            ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");  
12            CONNECTED = 1;  
13            break;  
14        case MQTT_EVENT_DISCONNECTED:  
15            ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");  
16            CONNECTED = 0;  
17            break;  
18        case MQTT_EVENT_PUBLISHED:  
19            ESP_LOGI(TAG, "MQTT_EVENT_PUBLISHED, msg_id=%d", event->msg_id)  
20            ;  
21            break;  
22        case MQTT_EVENT_DATA:  
23            ESP_LOGI(TAG, "MQTT_EVENT_DATA");  
24            printf("TOPIC=.%s\r\n", event->topic_len, event->topic);  
25            printf("DATA=.%s\r\n", event->data_len, event->data);  
26            break;  
27        case MQTT_EVENT_ERROR:  
28            ESP_LOGI(TAG, "MQTT_EVENT_ERROR");  
29            if (event->error_handle->error_type ==  
30                MQTT_ERROR_TYPE_TCP_TRANSPORT) {  
31                ESP_LOGI(TAG, "Last errno string (%s)", strerror(event->  
32                    error_handle->esp_transport_sock_errno));  
33            }  
34            break;  
35        default:  
36            ESP_LOGI(TAG, "Other event id:%d", event->event_id);  
37            break;  
38    }  
39 }
```

Listing 30: Event Handler de la pile IP Netif

La configuration d'un gestionnaire d'événements pour un client MQTT sur l'ESP32 se fait en enregistrant une fonction de rappel avec `esp_mqtt_client_register_event`. Cette fonction, nommée `mqtt_event_handler`, est appelée pour gérer divers événements MQTT. Elle est configurée pour répondre à tous les types d'événements MQTT.

Dans le gestionnaire d'événements, différents cas sont traités en fonction de l'`event_id`.

Par exemple, lors de la connexion au broker MQTT, un message est enregistré et une variable `CONNECTED` est mise à jour.

En cas de déconnexion, cette variable est réinitialisée.

D'autres événements, comme la publication de messages ou la réception de données, sont également gérés avec des messages de journalisation appropriés.

En cas d'erreur, des détails supplémentaires sont enregistrés pour faciliter le diagnostic.

Ce gestionnaire permet de surveiller et de réagir efficacement aux différents états et événements du client MQTT.

3.10 Capteur MQ135

Le module MQ Sensor comporte un capteur MQ135 qui permet de mesurer la qualité de l'air intérieur, il permet de détecter le Benzène, l'Ammoniaque, la fumée, le sulfure et la pollution atmosphérique.

Ce module a une sortie analogique permettant de mesurer la qualité de l'air et une sortie analogique qui est déclenchée quand un seuil est dépassé.



Figure 32: Capteur MQ135

3.10.1 Gaz détectés :

- Ammoniaque (NH_3)
- Sulfure d'hydrogène (H_2S)
- Dioxyde de carbone (CO_2)
- Benzène (C_6H_6)
- Alcool ($\text{C}_2\text{H}_5\text{OH}$)
- Dioxyde d'azote (NO_2)
- fumée

Donnée	Valeur
Tension d'alimentation	5V
Intensité	180mA
Puissance consommée	0.9W
Températures de fonctionnement	-10°C à 45°C
Plage de mesure	10ppm à 10000ppm
Tolérance	±15%

4 Conclusion

Travailler au sein d'une structure aussi importante et diversifiée que Hager Group m'a offert une expérience enrichissante et formatrice. J'ai eu l'opportunité d'explorer et de mettre en pratique des compétences variées, allant de la gestion du parc informatique à la communication entre les PC de production et les automates. Cette expérience m'a permis de comprendre l'importance de l'intégration entre les technologies de l'information (IT) et les technologies opérationnelles (OT), élément indispensable au bon fonctionnement d'une entreprise industrielle.

L'apprentissage en milieu industriel m'a inculqué la rigueur et la précision nécessaires pour maintenir une haute disponibilité des services informatiques, essentielle pour soutenir les opérations de production. J'ai appris à travailler de manière méthodique et à anticiper les besoins et les défis potentiels, tout en assurant la sécurité et l'efficacité des systèmes.

L'aboutissement de cette expérience, à travers le projet E6, m'a permis de synthétiser et d'appliquer les compétences acquises tout au long de ma formation. Ce projet m'a donné l'occasion de me familiariser avec des technologies avancées telles que la conteneurisation, la gestion de conteneurs avec Kubernetes, la création de pages web avec Node-Red, et la gestion de bases de données. De plus, j'ai pu explorer la programmation embarquée avec des microcontrôleurs ESP32 et le versioning avec Git et GitHub, tout en renforçant mes compétences en sécurité réseau.

Cette expérience chez Hager Group s'est avérée extrêmement bénéfique, me permettant de développer des compétences techniques solides tout en acquérant une compréhension approfondie des exigences et des dynamiques d'un environnement industriel. Je suis convaincu que ces acquis seront précieux pour mon avenir professionnel et me permettront de contribuer efficacement à des projets complexes et innovants.